# Analyzing the Role of AI-Assisted Programming in Modern Software Development

## Task 1: Conceptual Understanding

### What is AI-Assisted Programming?

AI-Assisted programming is the use of artificial intelligence and machine learning models to help developers write, review, test and maintain software.

### Types of AI coding tools

Context-Aware Editors

AI Assistants/Autocompletes

Autonomous Agents/Vibe Coders

### Advantages in Software Development

Elevated Abstraction and "Vibe Coding"

Seamless Language and Domain Bridging

Rapid API Integration and Scaffolding

Accelerated Debugging and Refactoring

Autonomous Delegation

### Risks and limitations

Hallucinations and Silent Logic Errors

Security Vulnerabilities and Anti-Patterns

Architectural Myopia

Intellectual Property and Licensing Friction

Cognitive Dependency and Skill Regression

# Task 2: Practical Implementation

## Select a programming problem of moderate complexity

Vehicle Maintenance & Fuel Tracking System

An application designed for motorcycles owners to track their vehicle's performance and service history.

## Develop the solution using TWO approaches

## Version A:

```
import java.util.Scanner;


class Vehicle{
    private String name;
    private double eMilegae;
    private double sMileage;
    private double cOdometer;


    public Vehicle(String name, double eMilegae, double sMileage, double
cOdometer){
        this.name = name;
        this.eMilegae = eMilegae;
        this.sMileage = sMileage;
        this.cOdometer = cOdometer;
    }


    public void checkServiceStatus(){
        if(cOdometer > sMileage || cOdometer == sMileage){
            System.out.println("It is time to service your vehicle!");
        }else{
            System.out.println("Next service at:"+sMileage+"km");
        }
```

```
    }

    public double getCurrentOdometer(){
        return cOdometer;
    }

    public void updateOdometer(double Distance){

        cOdometer = cOdometer + Distance;
    }
}

class Trip{
    private String tName;
    private double sReading;
    private double eReading;
    private double sFuel;
    private double eFuel;

    public Trip(String tName,double sReading,double eReading,double
sFuel,double eFuel){
        this.tName = tName;
        this.sReading = sReading;
        this.eReading = eReading;
        this.sFuel = sFuel;
        this.eFuel = eFuel;
    }
    public double calculateDistance(){
        double distance = eReading-sReading;
        return distance;
```

```
    }
    public double calculateFuelConsumed(){
        double uFuel = sFuel-eFuel;
        return uFuel;
    }
    public double calculateFuelEfficiency(){
        double distance = calculateDistance();
        double uFuel = calculateFuelConsumed();
        double fConsumed = distance/uFuel;
        return fConsumed;
    }
}



    public class VehicleSystem{
        public static void main(String[] args){
            Scanner scan = new Scanner(System.in);

            System.out.println("Enter your MotorCycle Name:");
            String name = scan.nextLine();

            System.out.println("Enter vehicle target km/l");
            double eMileage = scan.nextDouble();

            System.out.println("Enter the next service mileage (km):");
            double sMileage = scan.nextDouble();

            System.out.println("Enter the current odometer reading
(km):");
            double cOdometer = scan.nextDouble();
```

```
            Vehicle myScooter = new
Vehicle(name,eMileage,sMileage,cOdometer);


            int tripChoice = 1;


            while(tripChoice == 1){
                System.out.println("Enter your Trip Name:");
                String tName = scan.next();


                System.out.println("Enter the start odometer reading
(km):");
                double sReading = scan.nextDouble();


                System.out.println("Enter the end odometer reading
(km):");
                double eReading = scan.nextDouble();


                System.out.println("Enter the starting fuel amount(l)");
                double sFuel = scan.nextDouble();


                System.out.println("Enter the ending fuel amount(l)");
                double eFuel = scan.nextDouble();


                Trip currentTrip = new
Trip(tName,sReading,eReading,sFuel,eFuel);


                double distance = currentTrip.calculateDistance();
                double efficiency = currentTrip.calculateFuelEfficiency();
```

```
                System.out.println("Fuel efficiency for " + tName + ":" +
efficiency + "km/l" );


                myScooter.updateOdometer(distance);
                myScooter.checkServiceStatus();


                System.out.println("Do you want to log another trip (1=
Yes ,0 = No)");
                tripChoice = scan.nextInt();




            }
            System.out.println("System shutting down.");
        }
    }
```

## Version B:

```
import java.util.Scanner;


// Vehicle Class
class Vehicle {
    private String name;
    private double targetFuelEfficiency;
    private double nextServiceMileage;
    private double currentOdometer;


    public Vehicle(String name, double targetFuelEfficiency,
                   double nextServiceMileage, double currentOdometer) {
        this.name = name;
        this.targetFuelEfficiency = targetFuelEfficiency;
```

```
        this.nextServiceMileage = nextServiceMileage;
        this.currentOdometer = currentOdometer;
    }

    public void updateOdometer(double distance) {
        currentOdometer += distance;
    }

    public void checkServiceStatus() {
        if (currentOdometer >= nextServiceMileage) {
            System.out.println("SERVICE ALERT!");
            System.out.println("Current Odometer: " + currentOdometer + "
km");
            System.out.println("Next Service Mileage: " +
nextServiceMileage + " km");
        } else {
            System.out.println("Service Status: OK");
            System.out.println("Remaining until service: "
                    + (nextServiceMileage - currentOdometer) + " km");
        }
    }

    public double getTargetFuelEfficiency() {
        return targetFuelEfficiency;
    }

    public String getName() {
        return name;
    }

    public double getCurrentOdometer() {
```

```
        return currentOdometer;
    }
}

// Trip Class
class Trip {
    private String tripName;
    private double startingOdometer;
    private double endingOdometer;
    private double startingFuel;
    private double endingFuel;

    public Trip(String tripName,
                double startingOdometer,
                double endingOdometer,
                double startingFuel,
                double endingFuel) {

        this.tripName = tripName;
        this.startingOdometer = startingOdometer;
        this.endingOdometer = endingOdometer;
        this.startingFuel = startingFuel;
        this.endingFuel = endingFuel;
    }

    public double calculateDistance() {
        return endingOdometer - startingOdometer;
    }

    public double calculateFuelConsumed() {
```

```
        return startingFuel - endingFuel;
    }

    public double calculateFuelEfficiency() {
        double fuelConsumed = calculateFuelConsumed();

        if (fuelConsumed <= 0) {
            return 0;
        }

        return calculateDistance() / fuelConsumed;
    }

    public String getTripName() {
        return tripName;
    }
}

// Main Class
public class VehicleMaintenanceSystem {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        // Vehicle Initialization
        System.out.println("=== Vehicle Initialization ===");

        System.out.print("Enter motorcycle name: ");
        String name = scanner.nextLine();
```

```
System.out.print("Enter target fuel efficiency (km/L): ");
double targetFuelEfficiency = scanner.nextDouble();

System.out.print("Enter next service mileage (km): ");
double nextServiceMileage = scanner.nextDouble();

System.out.print("Enter current odometer reading (km): ");
double currentOdometer = scanner.nextDouble();

Vehicle vehicle = new Vehicle(
        name,
        targetFuelEfficiency,
        nextServiceMileage,
        currentOdometer);

boolean running = true;

while (running) {

    scanner.nextLine(); // consume newline

    System.out.println("\n=== Log New Trip ===");

    System.out.print("Enter trip name: ");
    String tripName = scanner.nextLine();

    System.out.print("Enter starting odometer: ");
    double startOdometer = scanner.nextDouble();
```

```
            System.out.print("Enter ending odometer: ");
            double endOdometer = scanner.nextDouble();

            System.out.print("Enter starting fuel (L): ");
            double startFuel = scanner.nextDouble();

            System.out.print("Enter ending fuel (L): ");
            double endFuel = scanner.nextDouble();

            Trip trip = new Trip(
                    tripName,
                    startOdometer,
                    endOdometer,
                    startFuel,
                    endFuel);

            double distance = trip.calculateDistance();
            double fuelConsumed = trip.calculateFuelConsumed();
            double efficiency = trip.calculateFuelEfficiency();

            System.out.println("\n=== Trip Summary ===");
            System.out.println("Trip Name: " + trip.getTripName());
            System.out.println("Distance Travelled: " + distance + " km");
            System.out.println("Fuel Consumed: " + fuelConsumed + " L");
            System.out.printf("Fuel Efficiency: %.2f km/L%n", efficiency);

            if (efficiency >= vehicle.getTargetFuelEfficiency()) {
                System.out.println("Fuel efficiency is meeting the
target.");
```

```
            } else {
                System.out.println("Fuel efficiency is below the
target.");
            }

            // Update vehicle odometer
            vehicle.updateOdometer(distance);

            System.out.println("\n=== Service Check ===");
            vehicle.checkServiceStatus();

            System.out.print("\nDo you want to log another trip? (Y/N):
");
            char choice = scanner.next().charAt(0);

            if (choice == 'N' || choice == 'n') {
                running = false;
            }
        }

        System.out.println("\nThank you for using the Vehicle Maintenance
& Fuel Tracking System.");

        scanner.close();
    }
}
```

I generate Version B using ChatGpt and this is the prompt that I use:

Act as a Senior Software Engineer.I need to build a console based Vehicle Maintenance & Fuel Tracking System in Java.The Entire program must be contained within a single Java

file.The class containing the main method must be public.All other calsses must be non-public.

In Vehicle Class : Should track the motorcycle's name,target fuel efficiency (km/l),next service mileage and current odometer reading.It needs a method to update the overall odometer and a method to print an alert if the current odometer has surpassed the next service mileage.

In Trip Class: Should track a specific trip's name,starting odometer,ending odometer,starting fuel(in liters) and ending fuel.It needs methods to calculate the distance travelved,fuel consumed and the real-time fuel effciency(km/l).

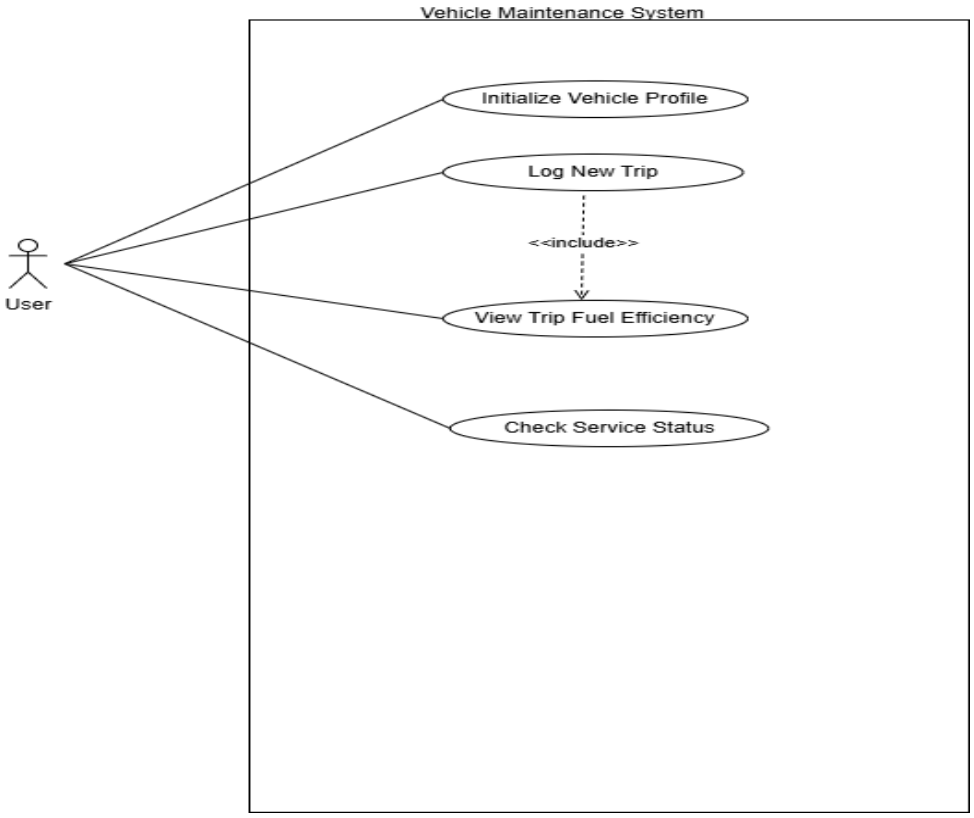
In Main Loop:The main method should ask the user to intialize their vehicle and then enter a while loop that allows them to log mulitple trips consecutively printing the trip's fuel efficiency and checking the service status after each entry.

## 3. Compare both implementations based on:

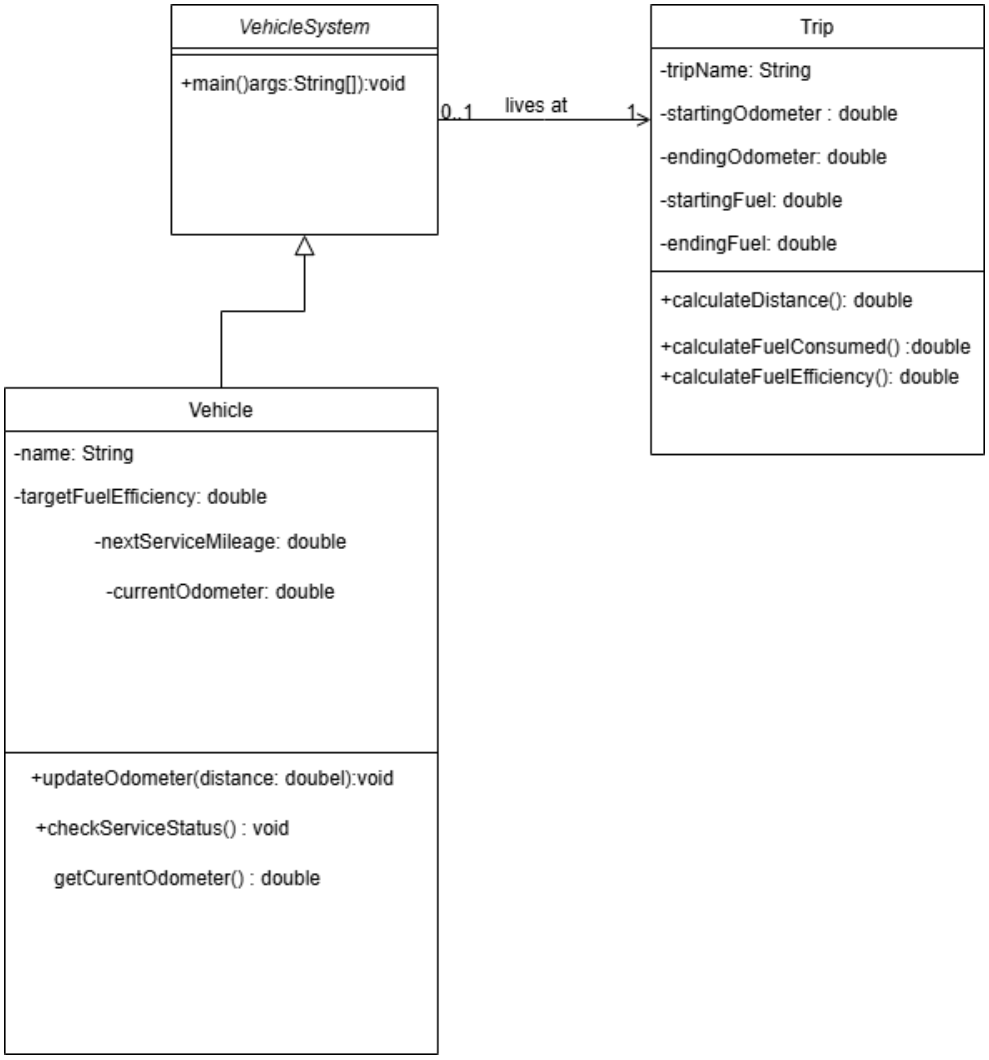
| | Version A | Version B |
|---|---|---|
| Development time | 3-4 hours<br>(Slower) | 5-10 minutes<br>(Fast) |
| Code quality | Good baseline OOP structure.<br>Logic is basic | Highly optimized |
| Error handling | None of error. | Much more robust |
| Readability and maintainability | Easy to read because it's simple | Professional |

**4.Include relevant UML diagrams to improve understanding of the selected problem and system design. Required UML Diagrams**

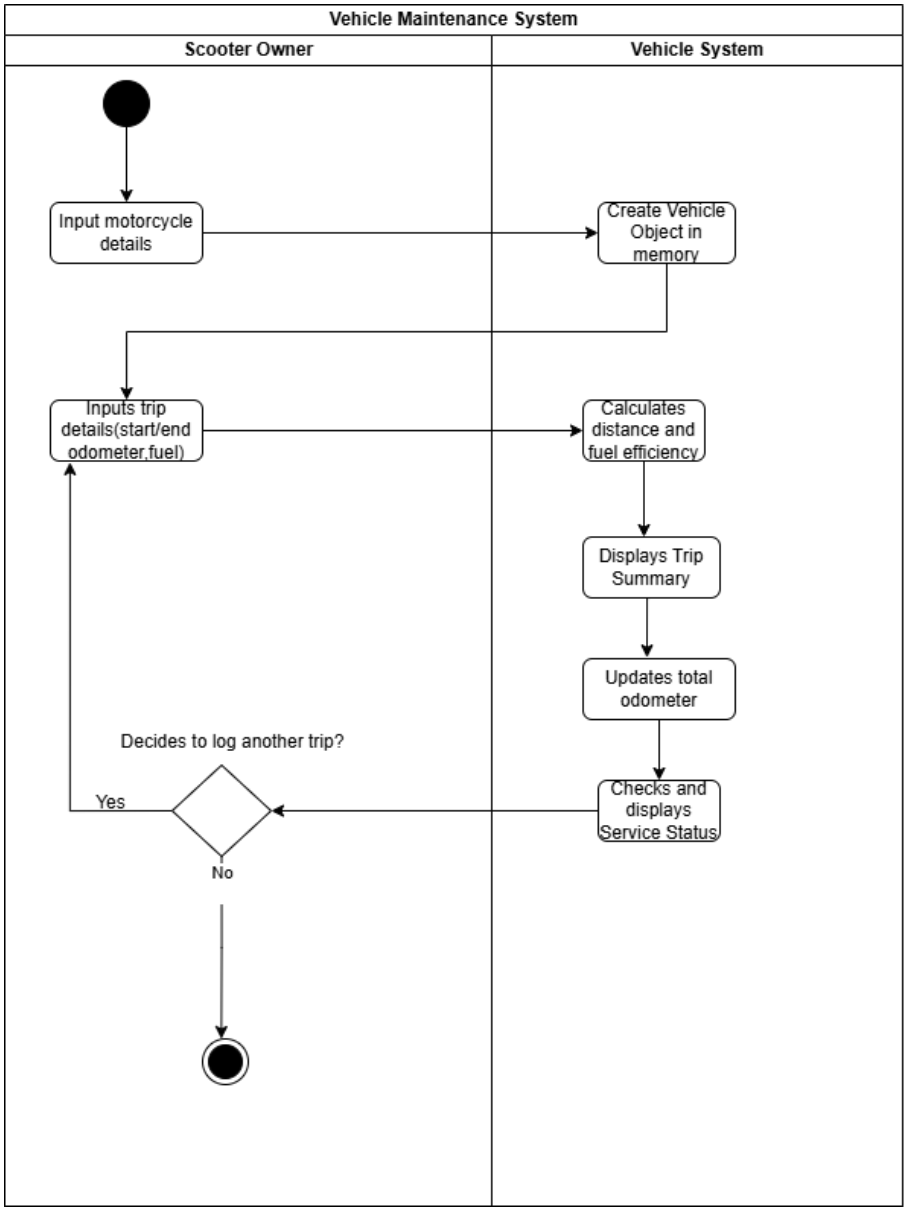
**Use Case Diagram:**



## Class Diagram:



## Activity Diagram:



# Task 3: Critical Analysis

## 1.How AI changed my coding approach

Using AI fundamentally shifted my coding approach from manual syntax construction to prompt engineering and system design. As you can see from my code comparison, the AI output is very much dependent on the initial instructions given. When writing Version A manually, my main concern was to get the basic OOP structure and running and ensure that the Java syntax was compiling. But in the process of creating Version B, I severed as a code reviewer. The AI handled the complex logic on its own. I focused on testing the AI's results , observing the automatic design decisions it took and refining my prompts to guide the system toward accurately monitoring the vehicle's performance.

## 2.Whether AI improved or reduced my understanding

Actually, working with AI improved my general understanding of system design because the process is quite similar to instructing a child, the output depends entirely on the clarity of the instructions I gave a bad, vague command like “Give me the JAVA code for a Vehicle System design”, the AI would give generic, misaligned and often unusable output. To get the high quality code required for Version B, I had to break down the problem into precise, detailed instructions for the Object Oriented structure and mathematical logic. The requirement to clearly state the classes, inputs and desired behaviors for the AI helped to understand my own understanding of the programming concepts. I had to structure the entire program in my head before the AI could write it.

## 3.In which cases AI should NOT be used

There are specific scenarios in both educational and industry where AI assistance should be strictly avoided.Absolute beginners starting to learn the basics of programming should not use AI in an educational context.If a student uses AI before they have learned the basic JAVA syntax nad logic they do not acquire the necessay problem solving skills.Most importantly, without a baseline understanding of the language a developer cannot meaningfully verify or debug the AI's output if it hallucinates incorrect logic.Moreover in a professional context, AI tools should never be used when dealing with highly sensitive company data,confidential user data. It is a serious security vulnerability and a violation of coporate data privacy rules to paste sensitive data or internal logic into public AI models or external APIs.

### 4.Ethical considerations in AI-generated code

There are many ethical issues that arise for software development due to AI. Accountability and the safety of systems developed with the use of AI are importance. The vehicle maintenance system project has shown us just how dangerous it is to trust AI generated code implicitly, there are risks that will not be immediately identifiable because of hidden bugs or faulty logic in the model used by the AI. A real world example could be an AI system generating an error in the Vehicle Management system by miscalculating an important service milestone. The result could be terrible if this happens and could result in physical injury or death due to failure of the brake system. This leads to the second major ethical question surrounding the development of software with AI, if something is created by an AI and there is a bug and something happens, who is responsible? The developer of the software, the provider of the Ai or the end user? Due to the ethical considerations that arise from AI generating code thee developer has a duty to act as the last line of defense against errors originating from the AI system by properly testing and validating all AI code.

## Task 4: Conclusion

### Final opinion on AI in programming education and industry

In the end Artificial Intelligence is really changing the way we do software engineering and programming education. For students, Artificial Intelligence is like a teacher who's always available to help explain tough ideas and make things easier. What we have seen in this project is that Artificial Intelligence cannot replace the basic learning process. A developer needs to understand basic programming principles to be able to use Artificial Intelligence generated code in a safe way to check for mistakes and to fix them.

Also, we need to give Artificial Intelligence models clear instructions, which shows how important it is to work on prompt engineering. The job of an engineer in the software industry is changing. While it is true that Artificial Intelligence might replace some tasks or jobs like entering data or making simple graphics, it is also creating new opportunities. The future of making software will depend on developers who can work with Artificial Intelligence systems and design systems using natural language prompts.

Really, we must use Artificial Intelligence in a way. If we use Artificial Intelligence a lot all the time it can hurt our ability to think creatively and solve problems. So the goal, for developers today is not to let Artificial Intelligence do all the thinking. To use it to do tasks that are repeated. If we can find a balance, we can use our creativity to focus on new ideas and design systems that are fair. Artificial Intelligence is a tool and we need to use it to make our work better not to replace our own thinking. By doing this we can make sure that Artificial Intelligence helps us than hurting us.